

權重是怎麼被改的

對應程式碼：`scratch/adamw.py`

分界線

```
model.backward(dlogits)      # 算出「該往哪修」 → 存進 dE、dW
opt.step()                   # 這一行才真的改動權重
```

反向傳播從不修改權重。這兩件事之間隔著一整張梯度表。

最直覺的版本：SGD

$$\theta \leftarrow \theta - \eta g$$

「減掉梯度乘上學習率」。

為什麼是減？梯度的定義是「往哪個方向動， L 會變大」：

$$g = \frac{\partial L}{\partial \theta}$$

我們要 L 變小，所以走反方向。這就是梯度下降的下降。

AdamW 做了什麼

$t \leftarrow t + 1$	
$\theta \leftarrow \theta - \eta \lambda \theta$	① decoupled weight decay（與梯度無關）
$m \leftarrow \beta_1 m + (1 - \beta_1) g$	② 一階動量：梯度的移動平均
$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$	③ 二階動量：梯度平方的移動平均
$\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}, \quad \hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$	④ bias correction
$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$	⑤ 真正的更新

跟 SGD 的差別只在 ⑤： g 被換成了 $\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$ 。

bias correction 在做什麼： m 、 v 都從 0 出發，前幾步會系統性偏小。除以 $1 - \beta_1^t$ 剛好把這個偏差補回來（ t 大了之後這個係數趨近 1，就沒作用了）。

關鍵性質：位移量幾乎與梯度大小無關

看 ⑤ 的分子分母：分子是梯度的平均，分母是梯度大小的平均。量級相同，相除之後大約落在 ± 1 ，於是

$$|\Delta\theta| \approx \eta$$

第一步時這件事是**精確**成立的。代入 $m_0 = v_0 = 0$ ：

$$m_1 = (1 - \beta_1)g \implies \hat{m}_1 = \frac{m_1}{1 - \beta_1} = g$$

$$v_1 = (1 - \beta_2)g^2 \implies \hat{v}_1 = \frac{v_1}{1 - \beta_2} = g^2$$

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \frac{g}{|g|} = \text{sign}(g) \implies \Delta\theta = \pm \eta$$

追蹤檔裡可以直接驗證：

	維度 1	維度 2	維度 3	維度 4
梯度 g	+0.244449	-0.286912	-0.239161	+0.236112
$\hat{m}/(\sqrt{\hat{v}} + \epsilon)$	+1.000000	-1.000000	-1.000000	+1.000000
位移 $\Delta\theta$	-0.000500	+0.000500	+0.000500	-0.000500
$ \Delta\theta /\eta$	1.0000	1.0000	1.0000	1.0000

(此處 $\eta = 5 \times 10^{-4}$ 。)

四個維度的梯度大小不同 ($0.236 \sim 0.287$)，位移卻**一模一樣**。

推論：總位移有上限

跑 T 步之後，任何一個參數的總位移上限大約是

$$\|\theta_T - \theta_0\| \lesssim T \cdot \bar{\eta}$$

實測驗算 (3000 步、cosine 從 3×10^{-3} 衰減)：

$$3000 \times 2.75 \times 10^{-4} \approx 0.825$$

而實測的平均位移落在 $0.63 \sim 0.86$ ——**正好卡在這個上限**。

差別只在**方向有沒有一致**：

	位移量	$\cos(\theta_0, \theta_T)$
高頻 token	接近上限	高（方向沒怎麼變）
中頻 token	接近上限	低（轉得最多）
從沒出現的 token	接近上限	中

位移量被 Adam 封頂，真正有差別的是「動得有沒有方向」。

高頻 token 出現在幾百萬個不同上下文，每個上下文想把它拉往不同方向，加起來互相抵消——所以它走得遠，但方向沒怎麼變。

decoupled weight decay

舊的 Adam 把 $\lambda\theta$ 加進梯度裡：

$$\theta \leftarrow \theta - \eta(g + \lambda\theta)$$

問題是這樣一來 weight decay 也會被 $\sqrt{\hat{v}}$ 正規化，強度變得跟梯度大小有關，很不直覺。

AdamW 把它拆出來獨立做（上面的 ①），強度固定是 $\eta\lambda$ 。這就是 **W** 的由來。

一個實務上的坑：`torch.optim.AdamW` 預設 $\lambda = 0.01$ ，而且會套用到**所有**參數，包括 embedding 和 norm 的 gain。

生產級訓練通常把這兩類排除——因為很少拿到梯度的列會被 weight decay 慢慢磨向 0。這份程式用 `no_decay` 參數控制，預設就排除了：

```
AdamW(model.params(), lr=..., no_decay=('.g', '.E'))
```

梯度裁剪

```
opt.clip_grad_norm(1.0)
```

把所有梯度串成一個超長向量，計算它的長度：

$$G = \sqrt{\sum_{\text{所有參數}} \|g\|^2}$$

若 $G > G_{\max}$ ，**整體**按比例縮小：

$$g \leftarrow g \cdot \frac{G_{\max}}{G + 10^{-6}}$$

注意是「整體」——各參數之間的相對比例不變。這跟逐元素裁剪（clip by value）不同，後者會扭曲梯度的方向。

用途是防止偶爾出現的異常大梯度（壞資料、數值不穩）一步把模型炸掉。

learning rate schedule

```
lr = cosine_lr(step, total, base_lr, warmup=100)
```

$$\eta(t) = \begin{cases} \eta_0 \cdot \frac{t}{T_{\text{warm}}}, & t < T_{\text{warm}} \\ \eta_0 \left[\rho + (1 - \rho) \cdot \frac{1 + \cos\left(\pi \frac{t - T_{\text{warm}}}{T - T_{\text{warm}}}\right)}{2} \right], & t \geq T_{\text{warm}} \end{cases}$$

其中 ρ 是最低點的比例（預設 0.1）。

- **warmup**：一開始權重是亂數，梯度方向不可靠，這時候用大 η 容易走歪。
- **cosine 衰減**：後期要小步微調，不然會在最低點附近震盪。

`traces/train_log.txt` 記了每一步的 η ，可以直接畫成曲線。